

Data Mining and Big Data Anlysis

数据挖掘及大数据分析

Chapter4：Data Preprocessing

数据预处理：02-数据集成

王东、张谦、丁玥

上海交通大学软件学院RFID与物联网实验室

2025年春季

Chapter 4: 数据预处理

- 概述
- 数据清洗（见数据预处理—数据清洗）
- 数据集成（见数据预处理—数据集成）
- 数据规约（见数据预处理—数据规约）
- 数据变换与数据离散化
- 小结



数据集成Data Integration

- 定义
  - 将数据从多个数据源合并到一致的存储coherent store
    - **实体识别问题**
    - **冗余和相关分析**
- 实体识别问题 **Entity identification problem**
  - 从各种数据源中识别出现实世界的实体
  - 模式集成 **Schema integration**
    - **e.g., A.cust-id ≡ B.cust-#**
    - 元数据集成
      - 每个属性的元数据包括名字、含义、数据类型和属性允许取值范围，以及处理空白、零或值的规则等用来帮助避免模式集成的错误
  - 属性匹配
    - 当一个数据库的属性与另一个数据库的属性匹配时，必须特别注意数据的结构。这旨在确保源系统中的函数依赖和参照约束与目标系统中的匹配。例如，在一个系统中 **discount**可能用于订单，而在另一个系统中，它用于订单内的商品。如果在集成之前未发现，则目标系统中的商品可能被不正确地打折
    - 同一个实体属性值可能不同，原因包括: 不同的表达方式、不同的取值范围，**e.g., metric vs. British units**

数据集成Data Integration

- 冗余和相关分析

—当多个数据库集成时，容易发生冗余

- 对象标识：不同数据库中属性或维名不一致
- 衍生数据 Derivable data: 一个属性可能由另一个或一组属性导出, e.g., 年收入

—冗余属性可以用相关分析（correlation analysis）检测

- 给定两个属性，这种分析可以根据可用的数据，度量一个属性能在多大程度上蕴涵另一个
  - 对于标称属性，使用X2 （卡方）检验
  - 对于数值属性，使用相关系数和协方差，评估一个属性的值如何随另一个变化

标称数据的相关检验：卡方测试（Chi-Square Test）

- 卡方测试是一种非参数统计检验，用于判断两个离散变量之间是否存在显著关系
  —例如，用卡方测试来分析“性别”和“是否喜欢某款游戏”是否相关
- 卡方(χ 2)统计量的公式为：

$$\chi^2 = \sum \frac{(O - E)^2}{E}$$

- (χ^2)：卡方统计量，表示实际观测值和期望值之间的差异程度。
- (O)：实际观测值（Observed Value）。
- (E)：期望值（Expected Value），计算公式为：

$$E = \frac{\text{行合计总计} \times \text{列合计总计}}{\text{总样本数}}$$

- 当χ 值越大，表明实际观测值与期望值之间的差距越大，两变量的关联性越强

卡方测试案例分析

问题：性别（男/女）与是否喜欢某款游戏

性别	喜欢	不喜欢	总计
男	250	200	450
女	50	1000	1050
总计	300	1200	1500

$$\chi^2 = \sum \frac{(O-E)^2}{E}$$

- (χ^2)：卡方统计量，表示实际观测值和期望值之间的差异程度。
- (O)：实际观测值（Observed Value）。
- (E)：期望值（Expected Value），计算公式为：

$$E = \frac{\text{行合计总计} \times \text{列合计总计}}{\text{总样本数}}$$

第一步：计算期望值

使用公式

$$E = \frac{\text{行合计总计} \times \text{列合计总计}}{\text{总样本数}}$$

计算：

- 男且喜欢：

$$E = \frac{450 \times 300}{1500} = 90$$

- 男且不喜欢：

$$E = \frac{450 \times 1200}{1500} = 360$$

- 女且喜欢：

$$E = \frac{1050 \times 300}{1500} = 210$$

- 女且不喜欢：

$$E = \frac{1050 \times 1200}{1500} = 840$$

期望值表格如下：

性别	喜欢	不喜欢
男	90	360
女	210	840

第二步：计算卡方统计量

$$\chi^2 = \frac{(250-90)^2}{90} + \frac{(200-360)^2}{360} + \frac{(50-210)^2}{210} + \frac{(1000-840)^2}{840}$$

数值数据的相关系数：冗余和相关分析

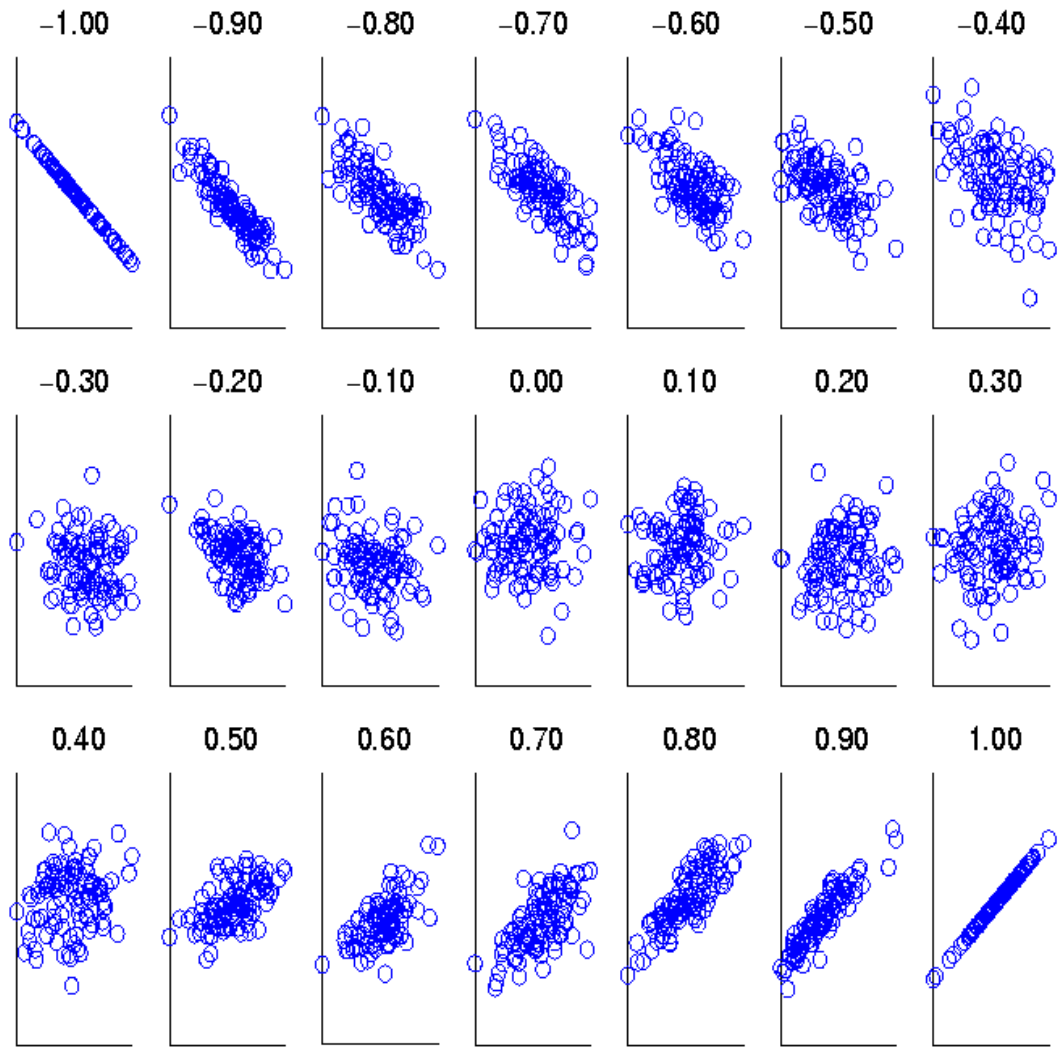
- 相关系数（又称皮尔森积矩系数Pearson's product moment coefficient）

$$r_{A,B} = \frac{\sum_{i=1}^{n}(a_i - \overline{A})(b_i - \overline{B})}{(n-1)\sigma_A\sigma_B} = \frac{\sum_{i=1}^{n}(a_i b_i) - n\overline{A}\overline{B}}{(n-1)\sigma_A\sigma_B}$$

其中n是元组的个数，$\overline{A}$ 和 $\overline{B}$ 分别是是A和B的均值，σ_A 和 σ_B 分别是A和B的标准差，$\Sigma(a_i b_i)$是AB叉积和

- If $r_{A,B} > 0$， A and B正相关（A随着B值的增加而增加），该值越大，相关性越强
- If $r_{A,B} = 0$， A and B独立
- If $r_{AB} < 0$， A and B负相关

Visually Evaluating Correlation



Scatter plots showing the similarity from –1 to 1.

数据集成

- 基本概念
- 数据集成的方式
- 数据集成的架构
- 数据集成的工具
- 数据集成技术发展趋势

数据集成：广义概念

- 在企业中，由于开发时间或开发部门的不同，往往有多个异构的、运行在不同的软硬件平台上的信息系统同时运行，这些系统的数据源彼此独立、相互封闭，使得数据难以在系统之间交流、共享和融合，从而形成了“信息孤岛”
- 随着信息化应用的不断深入，企业内部、企业与外部信息交互的需求日益强烈，急切需要对已有的信息进行整合，联通“信息孤岛”，共享信息
- **数据集成是一个过程，它涉及将来自不同数据源（如不同的数据库、应用程序和业务系统）的数据结合到一起，形成一个统一、一致的视图。这个过程包括数据的提取、清洗、转换和加载，通常用于构建数据仓库、数据湖或其他集成的数据存储解决方案**
  - 数据集成通过应用间的数据交换达到集成，主要解决数据的分布性和异构性问题，其前提是被集成应用必须公开数据结构，即必须公开表结构、表间关系、编码的含义等

数据集成是把不同来源、格式、特点性质的数据在逻辑上或物理上有机地集中，从而为企业提供全面的数据共享



数据集成主要应用场景：(1)数据仓库建设

- **场景描述**
  - —将企业分布在多个业务系统（如 ERP、CRM、财务系统等）的数据整合到统一的数据仓库中，为后续的 BI 报告和决策支持提供支持
- **需求特点**
  - —数据来源广泛，包括关系型数据库、API 数据、文件等
  - —需要清洗、转换和规范化数据
  - —数据量大，多为批处理

数据集成主要应用场景： (2)数据湖建设

- **场景描述**
  - —企业利用数据湖存储和管理多模态数据（结构化、半结构化、非结构化），如日志、图像、视频和 IoT 数据
- **需求特点**
  - —数据源多样，格式复杂（如 Avro、Parquet、JSON、CSV，DBF等）
  - —大量历史数据的批量导入与实时数据同步并存
  - —数据质量管理和数据版本管理

数据集成主要应用场景： (3)实时数据监控

- **场景描述**
  - —处理用户点击流数据、IoT 传感器数据、交易流水等实时数据流，支持实时监控、风险控制、告警和个性化推荐等场景
- **需求特点**
  - —数据吞吐量大，延迟要求低
  - —数据流式处理为主，部分批处理
  - —需要支持事件驱动架构

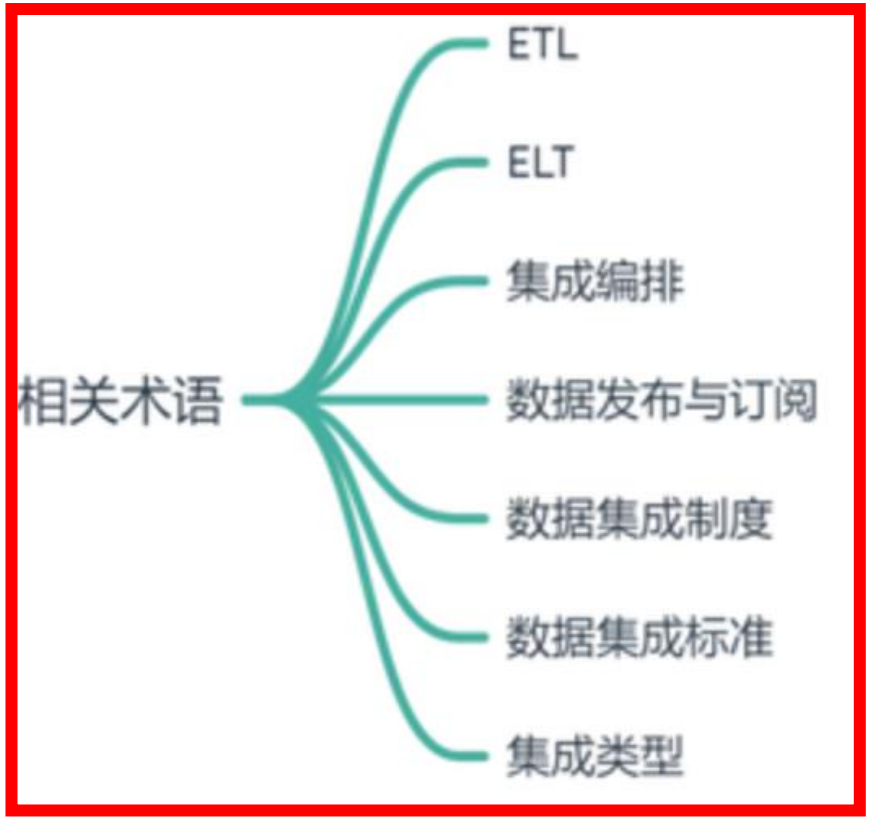
数据集成主要应用场景：(4)数据迁移

- **场景描述**
  - —系统升级或迁移时，需要将旧系统中的数据迁移到新系统或新架构（如从本地数据库迁移到云数据库）
- **需求特点**
  - —一次性大规模数据传输
  - —数据格式可能需要转换
  - —高度依赖数据完整性和准确性

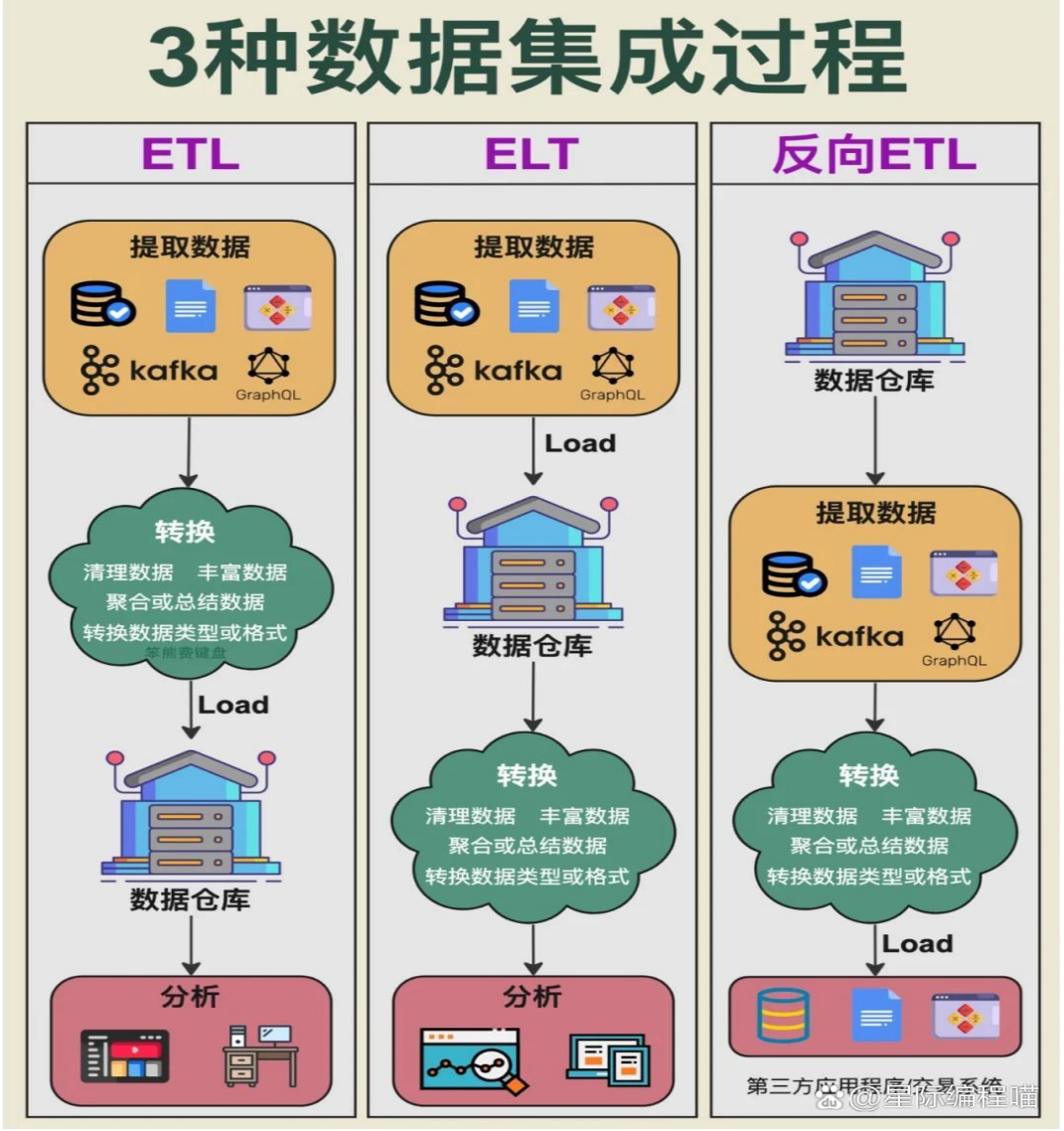
数据集成的知识体系：基本概念







数据集成的三种过程：ETL、ELT、Reverse ETL（反向ETL）



ETL

- 定义
  - —ETL代表提取（Extract）、转换（Transform）、加载（Load），是数据仓库中使用的一种过程，用于从各种数据源系统提取数据，对数据进行清洗、转换和整合，然后加载到目标数据仓库或数据湖中
    - 提取（Extract）：从原始数据源提取数据，这些源可能包括数据库、CRM系统、ERP系统、平面文件等
    - 转换（Transform）：对数据进行清洗和转换，以确保它们符合目标系统的要求。这可能包括数据清洗、去重、标准化、验证和应用业务规则
    - 加载（Load）：将转换后的数据加载到目标数据存储，如数据仓库、数据湖或其他分析数据库中
- ETL与数据集成的区别
  - —数据集成是一个更广泛的概念，涉及将数据从多个源系统集成到一起的任何技术或过程。除了ETL，还有其他方法如ELT（提取、加载、转换）、数据虚拟化和数据联邦等
  - —ETL特指一种特定的集成过程，即先提取数据，然后在中间层转换数据，最后加载到目标系统。这个过程通常是批处理的，而不是实时的

ELT

- 定义
  - ELT代表提取（Extract）、加载（Load）、转换（Transform），原始数据被提取出来并直接加载到目标数据存储（通常是数据湖或现代数据仓库），然后在目标系统内部进行转换和整理
- ELT与ETL的区别
  - **执行顺序**: 在ETL中，数据先在外部系统或中间层转换后再加载到目标系统。而在ELT中，数据先加载到目标系统，然后在目标系统内部进行转换
  - **性能和规模**: ELT通常更适合处理大规模的数据集，因为它可以利用现代数据仓库的高性能和弹性来处理转换。ETL可能需要额外的硬件和处理能力来处理大数据
  - **灵活性**: ELT提供了更高的灵活性，特别是当使用数据湖和云基础设施时。用户可以先将所有数据快速加载进去，然后根据需要对数据进行转换和分析
  - **工具和技术**: ETL和ELT可能使用不同的工具和技术，ETL通常依赖于传统的数据集成工具，而ELT则利用了数据湖和现代数据仓库的强大能力

集成编排

- 定义
- **数据集成编排是指对数据集成过程进行规划、管理和调度的行为，以确保数据在多个系统和服务之间高效、准确且可靠地流动**。编排涉及定义数据流的逻辑和顺序，管理数据的提取、转换和加载（ETL）过程，以及处理错误和依赖关系。目的是自动化并优化数据集成流程，确保数据及时到达正确的目的地，并以正确的格式和质量呈现。**编排通常依赖于工作流管理系统或集成平台，这些系统提供了工具和接口来设计、执行和监控数据集成工作流**
- 数据集成编排的特点
  - —自动化：自动化复杂的数据集成流程，减少人工干预
  - —调度：能够计划和调度数据集成任务的执行，例如在非高峰时段自动运行数据同步任务
  - —监控和日志：跟踪数据集成流程的执行情况，记录成功和失败的事件，便于故障排查和性能优化
  - —错误处理：自动处理失败的任务，例如重试或发送警报
  - —依赖管理：处理不同数据集成任务之间的依赖关系，确保它们按正确的顺序执行

数据发布与订阅

- 定义
  - **数据发布与订阅（Publish/Subscribe）是一种消息传递范式**，消息的发送者（发布者）不会直接将消息发送给特定的接收者（订阅者）。相反，发布的消息被分类到某个频道或主题，而订阅者表达了对一个或多个频道或主题的兴趣。当新消息发布到某个频道时，所有订阅了该频道的订阅者都会收到消息数据发布与订阅的特点
  - 解耦：发布者和订阅者之间不需要知道对方的存在，他们只与消息系统交互
  - 灵活性：订阅者可以根据需要订阅或取消订阅不同的消息频道或主题
  - 扩展性：新的订阅者或发布者可以很容易地添加到系统中，不会影响现有的订阅者或发布者
  - 即时性：订阅者可以实时接收到他们订阅的主题的更新

数据集成制度

- 定义
  - 数据集成制度是指一组规则、标准和流程，用来指导和管理一个组织内的数据集成活动。这些制度确保数据集成工作的一致性、效率和安全性，同时符合相关的法律法规和业务需求
- 案例
  - 第一章：制度概述
    - 目标和目的：确保数据集成活动支持组织战略，提高数据的质量和可用性
    - 适用范围：适用于所有涉及数据集成的部门和个人
  - 第二章：数据标准和质量
    - 数据格式标准：定义统一的数据格式和编码标准，例如ISO日期格式、统一的货币表示等
    - 数据质量要求：确保数据准确性、完整性和一致性。定期进行数据质量审查
  - 第三章：数据安全和合规
    - 数据访问控制：基于角色和责任分配数据访问权限
    - 数据加密和保护：对敏感数据进行加密处理，确保数据传输和存储安全
    - 合规性要求：确保数据集成活动符合所有相关的数据保护法律和行业规范
  - 第四章：数据集成流程管理
    - ETL流程：定义标准的数据提取、转换和加载流程
    - 数据审核和批准：在数据正式集成到主要系统前进行审核和批准

数据集成制度

- 第五章：技术和工具
  - 技术选择：指导如何选择和使用数据集成工具和平台
  - 技术培训和支持：为相关人员提供必要的技术培训和支持
- 第六章：监控和报告
  - 性能监控：监控数据集成流程的性能，确保高效运行
  - 定期报告：定期向管理层报告数据集成项目的进展和效果
- 第七章：持续改进
  - 反馈和评估：定期收集反馈，评估数据集成制度的有效性
  - 更新和改进：根据技术发展和业务需求的变化，持续更新和改进数据集成制度
- 第八章：实施和执行
  - 实施日期：规定制度的生效日期
  - 责任部门：指定负责本制度实施和监督的部门

数据集成标准

- 定义
  - 数据集成标准是一套规范和准则，用于指导如何正确、高效地整合来自不同数据源的数据。这些标准确保数据集成过程的一致性、可靠性和有效性
- 案例
  - 文本和字符编码
    - 标准：所有文本数据应使用统一的字符编码，推荐使用 UTF-8 编码
    - 实践：在数据提取和加载过程中，确保所有源系统和目标系统均能正确处理 UTF-8 编码的文本
  - 日期和时间格式：
    - 标准：所有日期和时间数据应使用国际标准 ISO 8601。格式为 YYYY-MM-DD（日期），YYYY-MM-DDTHH:MM:SS（时间戳）
    - 实践：在ETL过程中，将所有源数据中的日期和时间转换为 ISO 8601 标准格式
  - 数值格式
  - 货币表示
  - 文件格式
  - 数据集成工具的兼容性
    - 标准：选用的数据集成工具必须支持上述数据格式和编码标准
    - 实践：在选择数据集成工具时，验证其对上述标准的支持情况

集成类型

- 定义
  - 在现代企业架构中，集成不仅仅是关于数据。它涉及多种类型，以确保不同的系统、应用程序、服务和数据能够相互协作和通信
- 除了数据集成外，还有以下几种常见的集成类型：
  - **应用集成**
    - 应用集成关注于不同应用程序之间的交互和通信。它使得一个应用可以访问另一个应用的功能和数据。这通常通过APIs（应用编程接口）、消息队列和中间件等技术实现。应用集成是实现自动化工作流、提高生产率和确保应用间一致性的关键
  - **服务集成**
    - 随着SOA（服务导向架构）和微服务架构的流行，服务集成变得越来越重要。它涉及将独立的服务组合成复杂的应用。这通常通过RESTful API、SOAP服务、RPC（远程过程调用）等技术实现。服务集成支持更灵活的应用设计，使得各服务可以独立开发、部署和扩展
  - **流程集成**
    - 流程集成关注于不同业务流程之间的协调和集成。这涉及到跨多个应用和服务的工作流程的定义、执行和监控。流程集成常通过BPM（业务流程管理）工具和工作流引擎实现。它帮助企业优化和自动化跨部门和应用的业务流程
  - **用户界面集成**
    - 用户界面集成使得不同应用的前端可以更加协调一致，为用户提供无缝的体验。这可能涉及将来自不同应用的数据和功能集成到一个统一的界面中，或确保不同应用遵循相同的设计原则和样式指南

数据集成技术的发展历程

1. ETL 的起源与早期阶段（20世纪70-80年代）

- 数据库（数据仓库）自带工具
- 早期的 ETL 工具多为定制脚本和手动流程，依赖程序员编写代码完成抽取、转换和加载。工具稀缺，数据量有限，主要应用于简单的批量处理

2. 工具化和专业化阶段（20世纪90年代）

- 商业 ETL 工具出现，降低了 ETL 流程的复杂性，支持更多数据源和大规模数据处理
- 可视化 ETL 流程设计工具出现，使得非技术用户也能参与数据集成工作
- 代表工具：
  - —Informatica PowerCenter
  - —IBM DataStage
  - —Microsoft SSIS (SQL Server Integration Services)

数据集成技术的发展历程

3. 大数据驱动的转型（21世纪初）

- 随着互联网的兴起，数据量呈指数级增长，传统的 ETL 工具难以应对大规模非结构化数据。数据技术（如 Hadoop、Spark）的兴起改变了数据处理模式
- 出现 ELT（Extract-Load-Transform）模式，将数据加载到目标系统后再进行转换，利用目标系统（如数据仓库、MPP 数据库）的强大计算能力
- 代表性工具
  - —Apache Sqoop
  - —Apache NiFi
  - —DataX

数据集成技术的发展历程

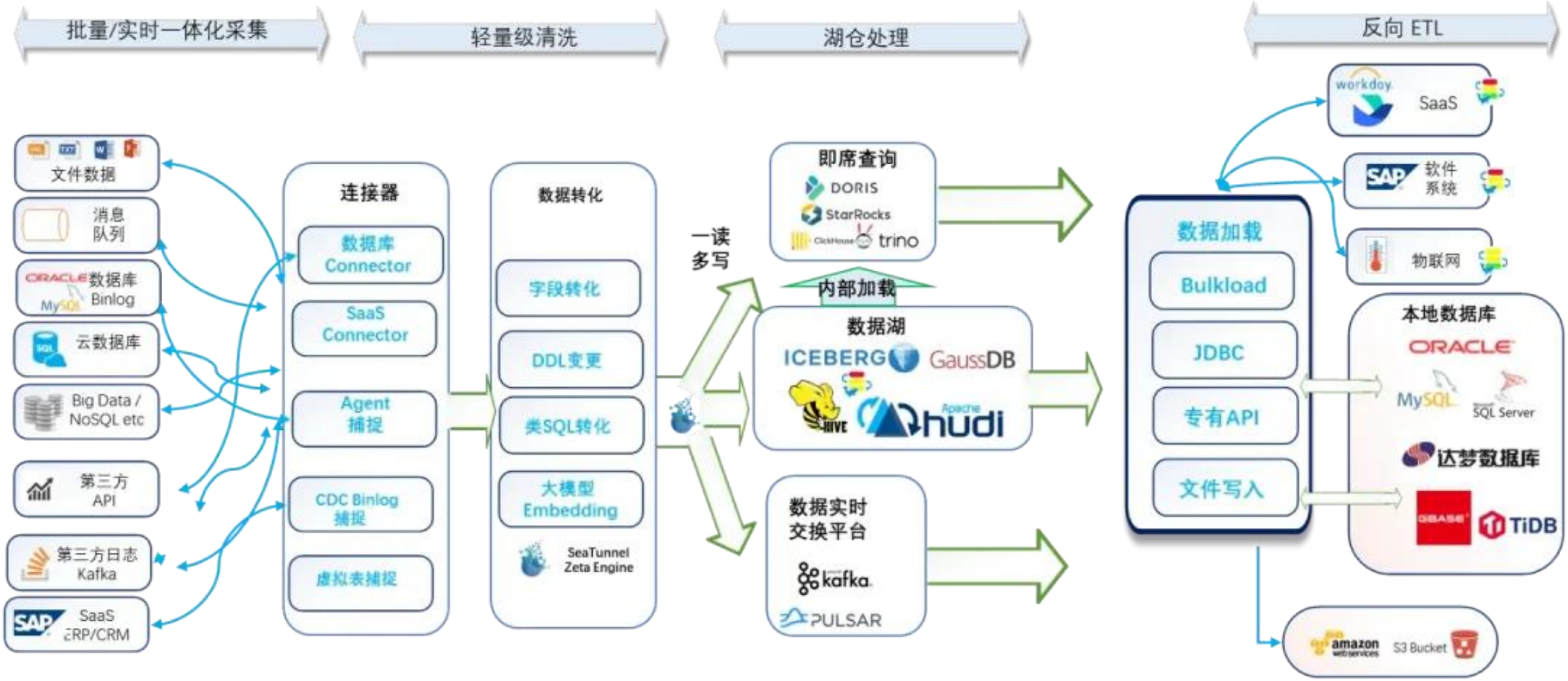
4. 云计算和现代数据栈的崛起（2010年代）

- 数据湖和实时计算兴起，批流一体要求CDC数据采集（ Change Data Capture，数据变更捕获）以及SaaS数据采集，ETL进化到ELT，数据仓库技术在云端的延伸（如 Redshift、Snowflake、Google BigQuery、S3等）。自助式 ETL 平台涌现，面向业务用户和数据分析师
- 代表性工具
  - Apache Flink
  - Apache SeaTunnel
  - Fivetran
  - WhaleStudio
  - Matillion

5. 2030-未来

- Tranform过程整合了大模型，数据处理扩展到音视频。大模型与AI融合到ETL过程中，AI集成为ETL的目标，也成为ETL中间的处理工具
- 动态ETL出现，自动数据处理而无需传统ETL任务，从而实现DataFabric

数据集成业界领先实践2024年



数据集成

- 基本概念
- 数据集成的方式
- 数据集成的架构
- 数据集成的工具
- 数据集成技术发展趋势

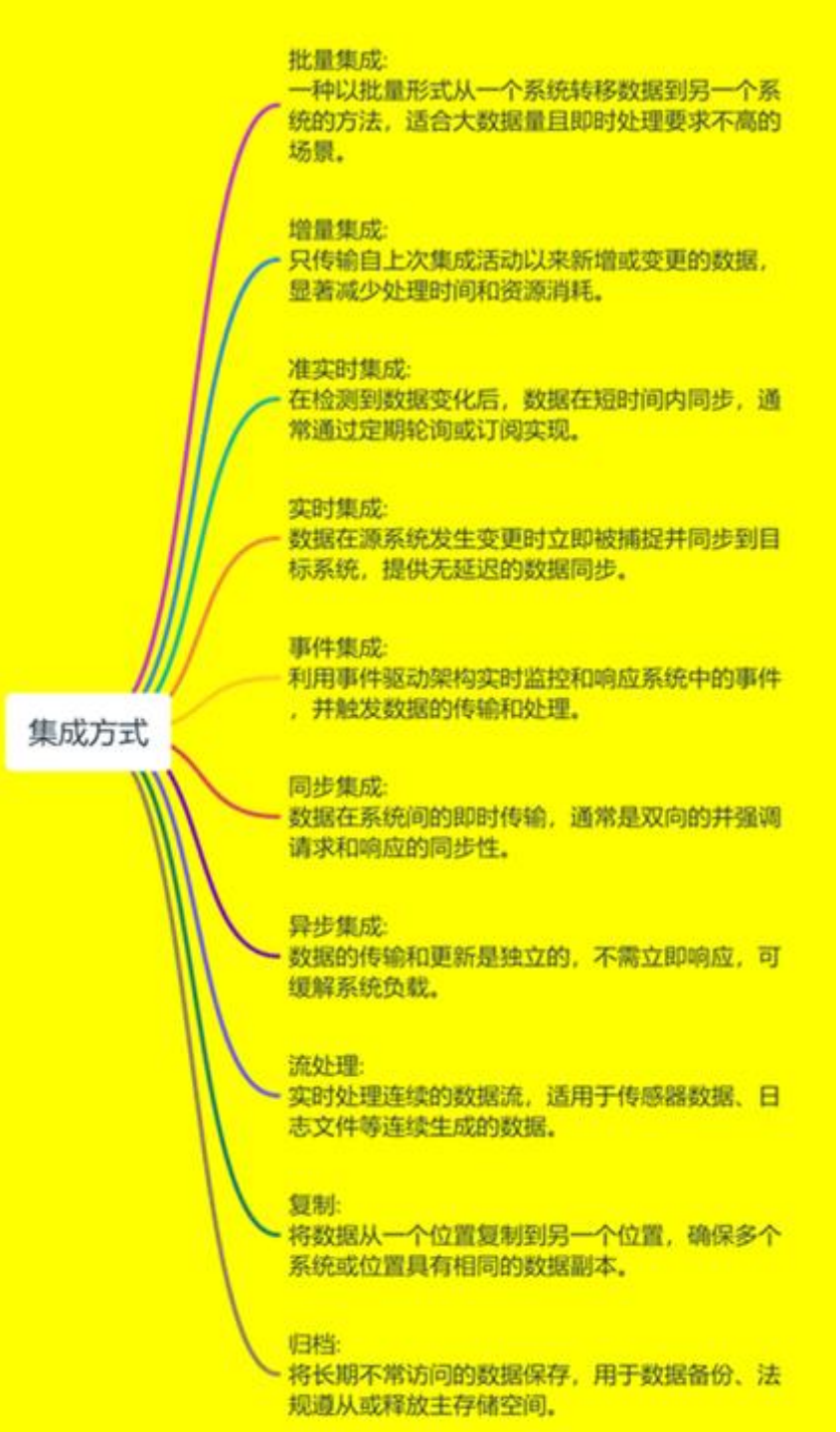
数据集成的知识体系


集成方式
批量集成
增量集成
准实时集成
实时集成
事件集成
同步集成
异步集成
流处理
复制
归档
集成架构
点对点架构
集中式架构
数据仓库架构
数据湖架构
服务导向架构
微服务架构
虚拟化架构
云化集成架构
相关术语
ETL
ELT
集成编排
数据发布与订阅
数据集成制度
数据集成标准
集成类型

数据集成的方式

1、批量集成

- 定义
  - —批量集成（Batch Integration）中**数据在定时或触发的基础上以批量形式全量处理，从一个系统转移到另一个系统**。这种方法通常在数据量大且对即时处理要求不高的场景中使用。批量集成能够处理大量数据，在非高峰时间执行，减少对业务运营的影响
- 典型引擎：
  - —hadoop mr、spark、Tez、Kettle、阿里云 DataWorks等



数据集成的方式

2、增量集成

- 定义

—只将自上次集成活动以来新增或变更的数据从一个系统转移到另一个系统的过程。它依赖于能够检测数据变化的机制，如时间戳、日志或触发器。与批量集成的全量处理相比，增量集成可以显著减少处理时间和资源消耗，提高系统响应速度。

数据集成的方式

3、准实时集成

- 定义
  - —系统在检测到数据变化后，不会立即但会在很短的时间内（通常是几分钟到几小时内）进行数据同步。这种方式通常通过定期轮询或订阅数据变更事件来实现，保证数据的较新状态，同时减轻系统的压力，避免了实时集成可能引起的高频率查询和数据冲突
- 典型引擎
  - —Storm、Spark Streaming等
- 案例
  - —一个大型零售公司，它拥有多个地理分布的仓库和销售点。该公司的库存管理系统需要确保所有位置的库存数据保持同步，以便于及时补货和处理订单。然而，由于库存变动频繁且数据量大，实时同步所有地点的库存变化既不经济也不实际
  - —为了有效管理库存，零售公司实施了准实时集成。在这种设置中，

数据集成的方式

4、实时集成

- 定义

  —实时集成指的是**数据在源系统发生变更的那一刻立即被捕捉并同步到目标系统**。这通常通过事件驱动的架构实现，其中数据变更会触发即时的处理和传输。实时集成保证了数据在系统间的同步几乎没有延迟，提供了最新的信息视图，支持快速决策和响应

- 典型引擎

  —Flink、Kafka Streams等

数据集成的方式

5、事件集成

- 定义

  —利用事件驱动架构（EDA）来实时监控和响应系统中发生的事件。一旦在源系统中检测到定义好的事件（如数据更新、新记录创建或特定条件的满足），就会触发数据的传输和相应的处理流程。这种方法不仅提供了实时性，还增加了系统间交互的灵活性和可扩展性

- 典型引擎

  —Kafka、RabbitMQ、EventBridge等

数据集成的方式

6、同步集成

- 定义

—**数据在系统间即时传输，响应于一个请求或事件**。当一个系统发出数据请求时，它等待并接收到最新的数据作为响应。强调请求和响应的同步性，即数据提供者在接到请求时，会立即处理并返回请求的数据。同步集成适用于事务性强、需要即时数据一致性的场景

—与实时集成相比，两者都是为了减少数据延迟，确保系统间数据的时效性和准确性。不过，**同步集成通常是双向的并需要即时响应，而实时集成则可能是单向的，只关注数据的实时更新**

数据集成的方式

7、异步集成

- 定义

—**数据的传输和更新不是在请求时立即发生，而是根据预设的条件或触发的事件进行**。在异步集成中，数据的发送和接收是独立的，发送方不需要等待接收方确认即可继续其他任务

—这种集成方式适用于不需要即时响应的场景，它可以缓解系统的即时负载，并允许更灵活的数据处理和传输时间安排

数据集成的方式

8、流处理

- 定义
  - —用于实时处理连续的数据流。这种处理方式能够即时分析和作出决策，通常用于处理不断生成的数据，如日志文件、传感器数据、金融交易等
  - —流处理系统通常具有高吞吐量和低延迟特点，能够在数据生成的瞬间进行捕获、分析和响应。流处理为实时集成提供了技术支持，特别是在需要处理大量连续数据流的情况下，例如从传感器或在线交易中实时收集数据
  - —但流处理并不总是需要数据集成，可以仅用于数据实时分析，不涉及将数据从一个系统转到另一个系统
- 典型引擎
  - —Kafka Streams、Flink、Storm、Spark Streaming等。

数据集成的方式

9、复制

- 定义

—**将数据从一个位置复制到另一个位置的过程，确保一个或多个系统或位置具有相同数据的副本**。这通常用于数据备份、灾难恢复、数据分发、负载均衡或提高数据访问性能。复制可以是同步的，也可以是异步的，并且可以涉及完整的数据复制或仅复制更改的数据（增量复制）

—数据复制通常仅关注数据的一致性和可用性。而数据集成通常涉及将来自多个源的数据结合在一起，创建一个统一的、全面的视图，不仅仅是复制数据，还包括数据的清洗、转换和合并

数据集成的方式

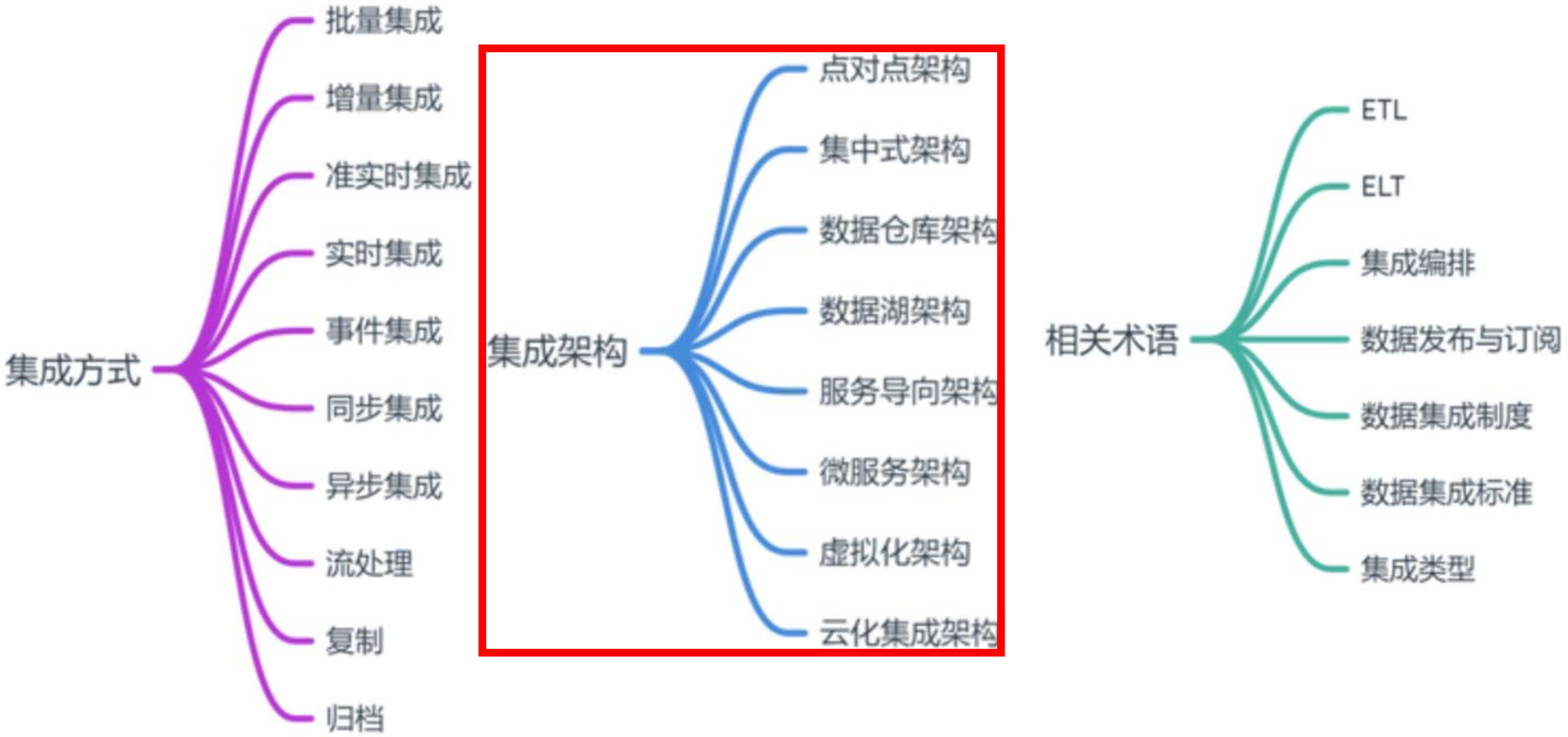
10、归档

- 定义
  - —将数据长期保存的过程，通常用于保留不再经常访问但仍具有保存价值的数据
  - —归档的目的可能是为了遵守法规要求、数据备份、历史记录保存或释放主存储空间。归档数据通常会被移动到更便宜或更专门的存储介质上，并且可能会经过压缩和优化以节省空间

数据集成

- 基本概念
- 数据集成的方式
- 数据集成的架构
- 数据集成的工具
- 数据集成技术发展趋势

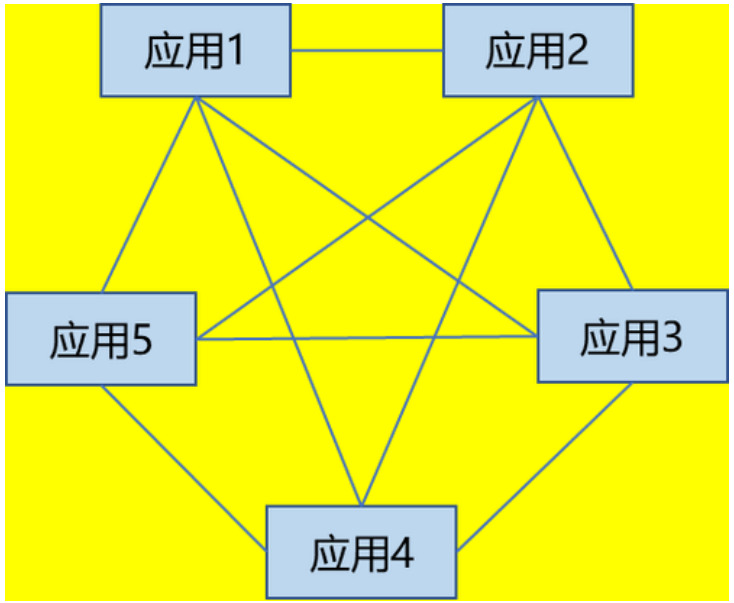
数据集成的知识体系


集成方式
批量集成
增量集成
准实时集成
实时集成
事件集成
同步集成
异步集成
流处理
复制
归档
集成架构
点对点架构
集中式架构
数据仓库架构
数据湖架构
服务导向架构
微服务架构
虚拟化架构
云化集成架构
相关术语
ETL
ELT
集成编排
数据发布与订阅
数据集成制度
数据集成标准
集成类型

数据集成：架构

1、点对点架构

- 点对点（Point-to-Point）架构中每个系统或应用直接与其他系统连接和交换数据。为了实现两个系统之间数据集成，会建立一个专门的通道或接口。每当需要集成额外的系统时，就需要建立新的连接
- 在只有少数系统需要集成时比较简单和直接。随着系统规模的扩大，管理和维护这些直接连接变得越来越复杂

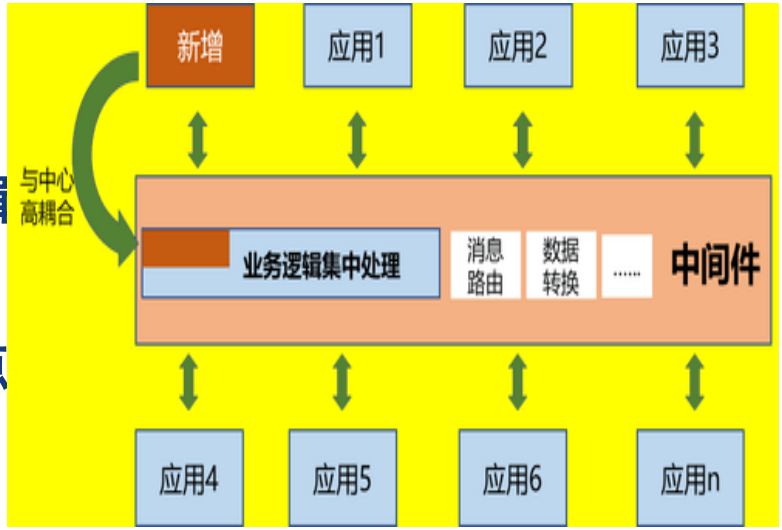




数据集成：架构

2、集中式架构

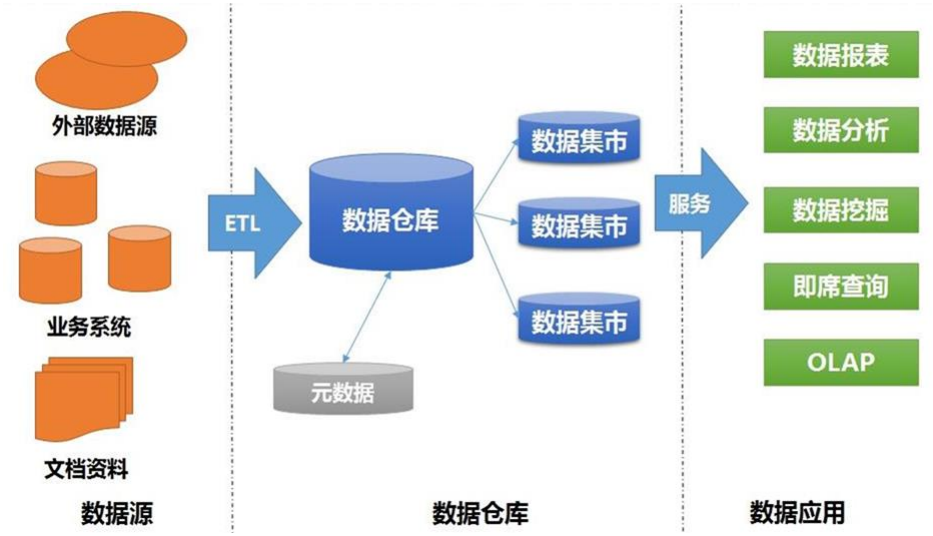
- 所有的数据交换和集成过程都通过一个中心点进行。这个中心点通常是一个集成服务器或中间件，如企业服务总线（ESB）或集成平台。简化了系统间的连接，每个系统不是与其他所有系统各自连接，而是与中心点通信，由中心点负责路由、转换和传递消息
- 集中式架构特点：
  - —集中的业务逻辑：中心点（如ESB）处理更多集成相关的业务逻辑
  - —集中的数据转换：中心点转换在不同系统间传递的数据格式
  - —集中的监控和管理：中心点作为所有集成活动的集中监控和管理点
  - —高耦合性：扩充系统不容易，对中心点及其他应用影响大



数据集成：架构

3、数据仓库架构

- 使用数据仓库作为集成数据的中心点，从各种源系统提取数据，然后通过清洗、转换（ETL过程）并加载到数据仓库中，这个集成过程使得来自不同源的数据被统一化和标准化，以支持复杂的查询和分析
  - 源系统：内部系统（如CRM、ERP、财务系统）或外部数据源
  - ETL或ELT工具：用于提取、转换和加载数据
  - 数据仓库：作为集成数据的中心存储库
  - 数据访问工具：如报告工具、分析工具和数据挖掘工具
- 数据仓库集成架构的特点
  - 集中式数据存储：所有关键业务数据存储在中央位置管理和分析
  - 历史数据存储：存储大量历史数据，以支持趋势分析和历史比较
  - 支持决策制定：为管理层提供了支持决策的统一数据视图
  - 数据质量管理：通过ETL过程中的清洗和转换步骤改进数据质量



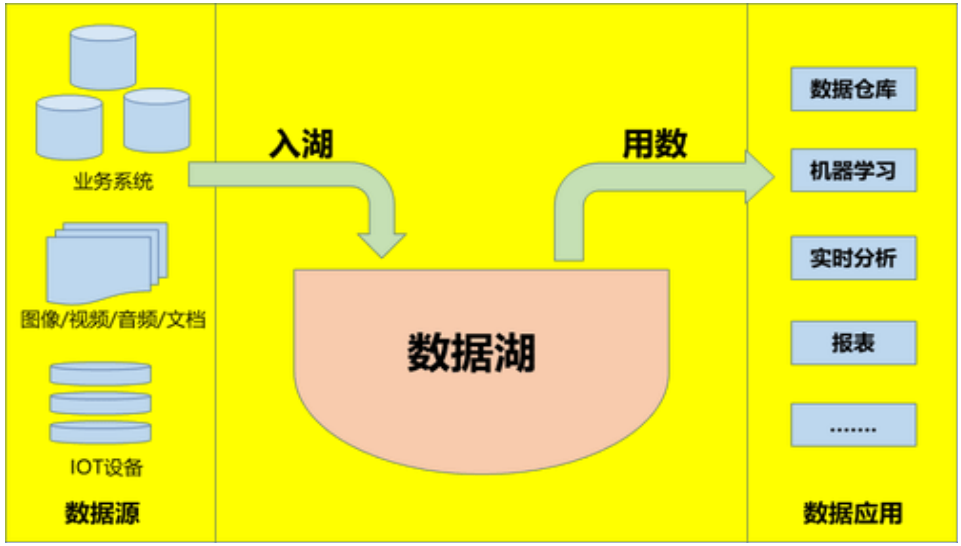
数据仓库架构与集中式架构区别

- 目的：侧重于为分析和决策支持提供历史数据的长期存储 & 侧重于数据的实时集成和通信
- 数据处理：数据经过ETL处理后存储，处理通常是批量进行的 & 数据通常在传输过程中实时处理和路由
- 用例：适用于需要复杂查询和深入分析的场景 & 用于需要实时数据集成和应用间通信的场景

数据集成：架构

4、数据湖架构

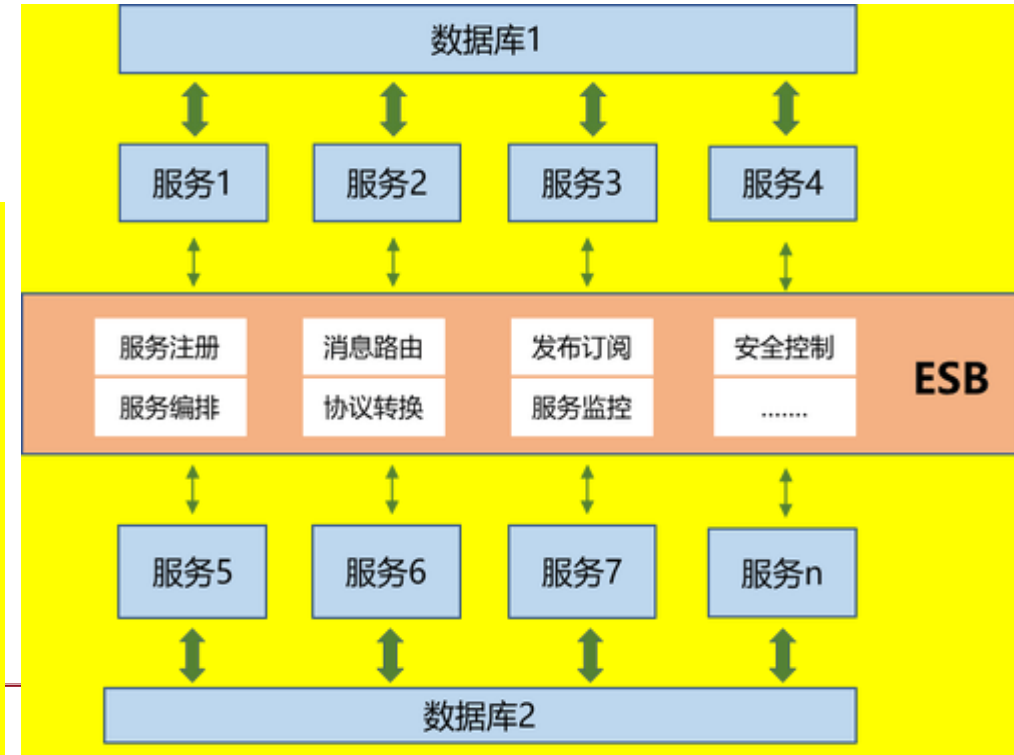
- 以数据湖为核心的数据管理架构，旨在存储和管理大量的原始数据，这些数据来自多个源且格式多样。数据湖允许存储结构化数据（如数据库中的行和列）、半结构化数据（如CSV、日志、XML、JSON）、非结构化数据（如电子邮件、文档、图片）和二进制数据。关键特点是，数据被存储在其原始格式中，直到需要时才进行处理和分析。数据湖中数据不仅仅是存储，还需要管理工具来组织、保护和访问数据：
  - 元数据管理：跟踪数据的来源、格式和使用情况
  - 数据治理：确保数据的质量和安全性
  - 数据访问：提供查询和分析工具来处理和视图数据
- 数据湖集成架构的特点
  - 灵活性：存储各种格式的数据，适应不断变化的数据需求
  - 可扩展性：适合大规模数据存储和处理，特别是用云平台时
  - 成本效益：相比传统数据仓库，存储原始数据通常更加经济
  - 即时性：能够存储实时流数据，支持快速分析和决策



数据集成：架构

5、服务导向架构（Service-Oriented Architecture，SOA）

- SOA的主要目标是提高业务灵活性和可重用性，通过构建一个能够快速响应变化并支持集成的多个组件和系统的架构。应用组件通过一个网络提供服务给其他组件，这些服务是松散耦合的，意味着服务的提供者和服务的消费者相互独立，它们之间的关系是通过定义良好的、统一的接口和合同实现的
- SOA允许不同的系统和应用通过标准化的服务接口共享和交换数据，这些服务可以是数据提取、转换、加载和查询等功能
  - 松散耦合：服务之间的依赖性最小化，易于替换和更新
  - 标准化接口：异构的服务通过协议转化过后的标准化接口提供，确保不同系统之间的兼容性
  - 重用性：服务可以在多个应用和业务流程中重用
  - 可扩展性：根据需要添加新的服务或升级现有服务，不影响其他组件



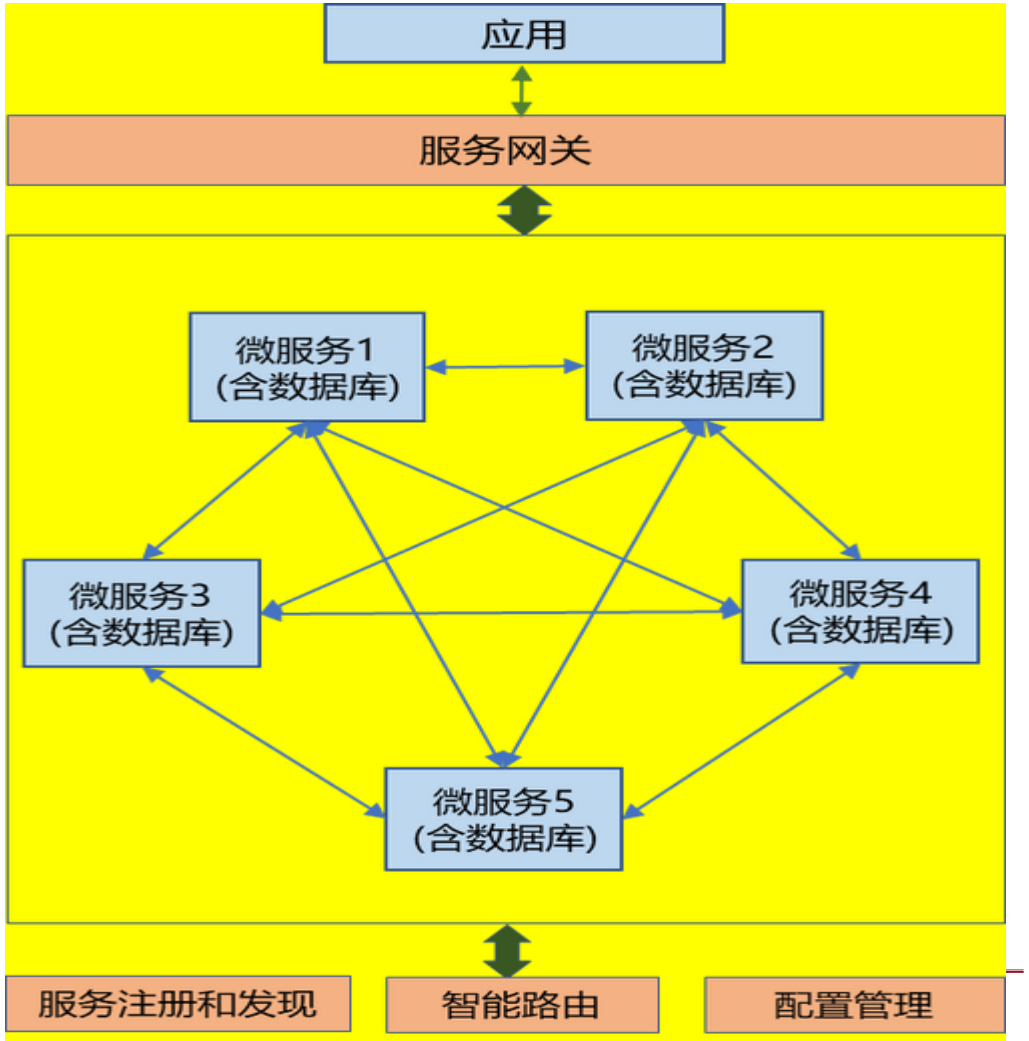
服务导向架构（SOA）与集中式架构区别

- 耦合程度：业务逻辑和数据转换在中心节点处理导致较高的耦合和集中的复杂性 & 服务更加自治，每个服务负责自己的逻辑和数据，提供了更高的灵活性和可维护性
- 灵活性和扩展性：中心点的任何修改可能影响到所有连接的系统 & 新增服务或修改现有服务比较容易，不会影响到其他服务
- 设计理念、架构目标和实现方式有本质的不同：虽然集中式架构和SOA在使用ESB进行集成时可能看起来有些相似，但集中式架构强调集中管理和简化集成，而SOA更注重服务的复用、自治和松耦合

数据集成：架构

6、微服务架构

- 将应用程序作为一组小的、独立的服务开发和部署，每个服务运行在其自己的进程中，并通常围绕特定的业务能力构建。这些服务可以用不同的编程语言编写，使用不同的数据存储技术，并且可以独立地部署和扩展。服务之间通过定义良好的API（通常是HTTP RESTful API）进行通信
- 微服务架构的特点
  - 独立性：每个微服务可以独立开发、部署、运行和扩展
  - 灵活性：团队可以选择最适合每个服务的技术栈
  - 可维护性：由于服务规模小，每个服务更易于理解和维护
  - 可扩展性：可以单独扩展某个服务，而不需要扩展整个应用
  - 敏捷性：团队可以更快地开发和部署新功能



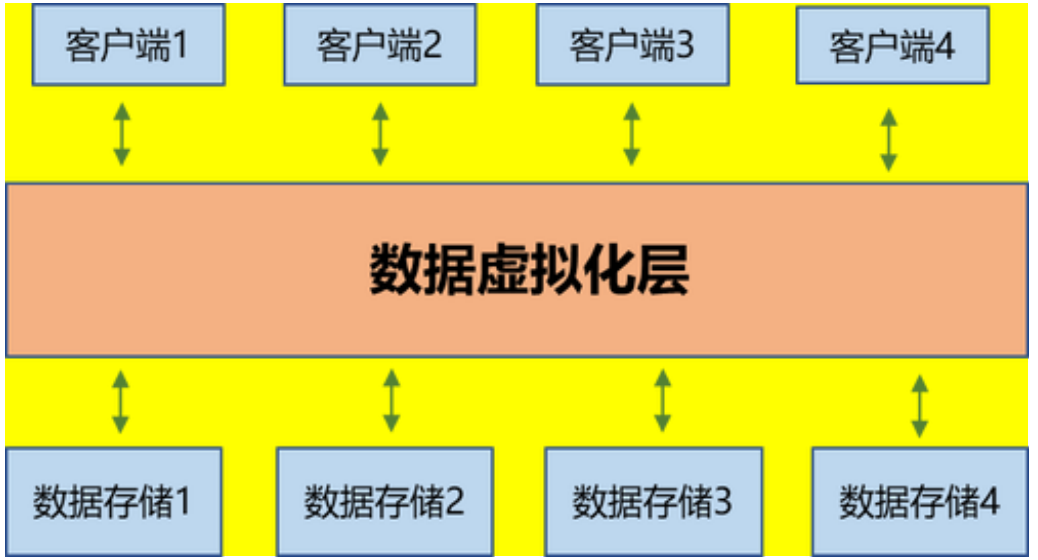
微服务架构与服务导向架构（SOA）的区别

- 设计哲学：微服务架构强调服务的独立性和小型化，每个服务都是自足的，有自己的数据和技术栈。而SOA强调服务的共享和复用，服务之间可能更加集中和依赖共享的资源和标准，比如两个服务共享数据库
- 服务通信：SOA采用了ESB作为服务间通信的关键组件，负责服务定义、服务路由、消息转换、消息传递，总体上是重量级的实现，性能开销非常大。微服务通过轻量级的通信协议进行松耦合通信（RESTful协议、RPC协议等），无须ESB这样的重量级实现
- 应用场景：SOA 适合于庞大、 复杂、异构的系统，采用兼容的方式进行各种存量异构系统间的集成；微服务更加适合于快速、轻量级、基于 Web 的互联网系统，这类系统业务变化快，需要快速尝试、快速交付

数据集成：架构

7、虚拟化架构

- **使用抽象层（通常是软件）来访问和管理来自不同数据源的数据，而无需将数据物理地移动或复制到一个集中位置。**这种架构允许用户和应用程序以统一和一致的方式查询、分析和操作分散在各处的数据。虚拟化层处理所有的数据转换、格式化和交付工作，使得底层数据源的复杂性对用户透明
  - 位置透明：用户可以访问和集成分布在不同地点的数据，而不需要知道数据实际存储的位置
  - 格式透明：虚拟化层可以处理不同数据源的数据格式和协议，提供一个统一的访问接口
  - 实时访问：用户可以实时访问最新数据，无需等待数据复制或转移
  - 成本效益：减少了数据移动和复制的需要，降低了存储和管理成本
  - 灵活性和可扩展性：容易添加新的数据源和适应变化的数据需求



数据集成：架构

8、云化集成架构

- **利用云服务和资源来实现数据和应用程序之间集成的架构。这种架构通常依赖于云提供商提供的平台和工具，例如云数据库服务、云存储、集成平台即服务（iPaaS）和API管理工具**
- 云化集成架构使企业能够更灵活、更高效地实现系统间的数据共享和业务流程协同，同时还可以利用云的弹性和可扩展性来处理大量数据和高并发请求
  - 可扩展性：能够根据需要动态地增减资源和服务
  - 灵活性：可以快速适应新的集成需求和变更
  - 成本效益：通常基于使用量付费，避免了大量的前期投资
  - 易于管理：云提供商通常提供管理工具和服务来简化运维工作

数据集成

- 基本概念
- 数据集成的方式
- 数据集成的架构
- 数据集成的工具
- 数据集成技术发展趋势

开源数据集成工具

- Apache InLong: 一站式、全场景的海量数据集成框架
- SeaTunnel: 下一代高性能、分布式、海量数据集成框架
- Chunjun:基于Flink的批流统一的数据同步工具
- BitSail：高性能数据集成引擎
- AirByte：开源的数据移动基础设施
- CloudCanal：数据同步、迁移工具
- Flume ：开源分布式、高可靠的流式日志采集系统
- Canal：数据库增量日志解析、采集工具
- Nifi：一个易于使用、功能强大且可靠的系统，用于处理和分发数据
- DataX：异构数据源离线同步工具

Apache SeaTunnel：专注于数据集成和数据同步

SeaTunnel主要解决数据集成领域的常见问题

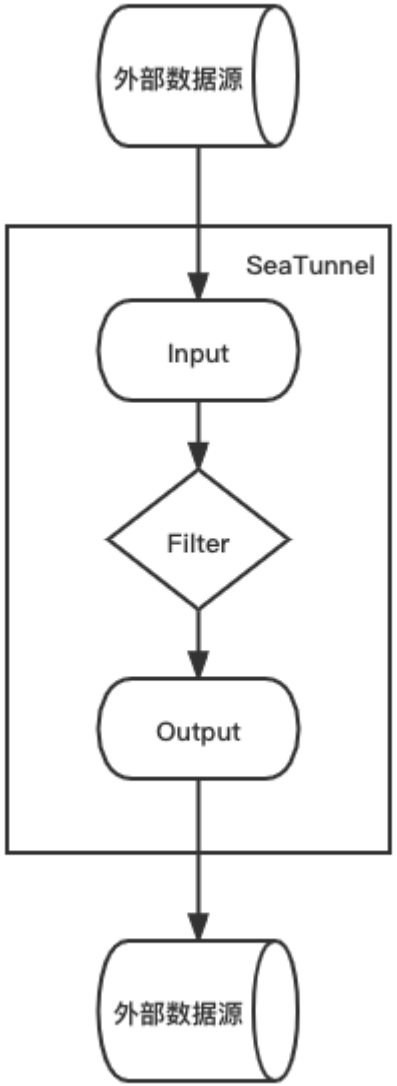
- **各种数据源**：有数百个版本不兼容的常用数据源。随着新技术的出现，出现了更多的数据源。用户很难找到一个工具，可以完全和快速地支持这些数据源
- **复杂的同步方案**：数据同步需要支持离线全同步、离线增量同步、CDC、实时同步、全库同步等多种同步场景
- **高资源需求**：现有的数据集成和数据同步工具往往需要庞大的计算资源或JDBC连接资源来完成海量小表的实时同步。这增加了企业的负担
- **缺乏质量和监控**：数据集成和同步过程经常会出现数据丢失或重复。同步过程缺乏监控，无法直观了解任务过程中数据的真实的情况
- **复杂的技术堆栈**：企业使用的技术组件不同，用户需要为不同的组件开发相应的同步程序来完成数据集成
- **管理和维护困难**：受限于不同的底层技术组件（Flink/Spark），离线同步和实时同步往往已经被分开开发和管理，这增加了管理和维护的难度

SeaTunnel: 支持的特性

- **丰富且可扩展的连接器**：SeaTunnel提供了一个不依赖于特定执行引擎的连接器API。基于此API开发的连接器（Source、Transform、Sink）可以在目前支持的SeaTunnel Engine、Flink、Spark等多种不同引擎上运行
- **连接器插件**：插件设计允许用户轻松开发自己的连接器并将其集成到SeaTunnel项目中。目前，SeaTunnel支持100多个连接器，并且数量正在激增
- **批次流整合**：基于SeaTunnel Connector API开发的连接器完美兼容离线同步、实时同步、全同步、增量同步等场景。它们大大降低了管理数据集成任务的难度
- **支持分布式快照算法，保证数据一致性**
- **多引擎支持**：默认情况下，SeaTunnel使用SeaTunnel引擎进行数据同步。SeaTunnel还支持使用Flink或Spark作为Connector的执行引擎，以适应企业现有的技术组件。SeaTunnel支持多个版本的Spark和Flink
- **JDBC复用，数据库日志多表解析**：SeaTunnel支持多表或全库同步，解决了跨JDBC连接的问题;支持多表或全库日志阅读和解析，解决了CDC多表同步场景需要处理日志重复阅读和解析的问题
- **高吞吐量、低延迟**：SeaTunnel支持并行阅读和写，提供稳定可靠的数据同步能力，具有高吞吐量、低延迟的特点
- **完美的实时监控**：SeaTunnel支持数据同步过程中每一步的详细监控信息，让用户轻松了解同步任务读写的数据数量、数据大小、QPS等信息
- **支持两种工作开发方法**：编码和画布设计。SeaTunnel网络项目https://github.com/apache/seatunnel-web提供作业、调度、运行和监控功能的可视化管理

Apache SeaTunnel：支持的插件

Input[数据源输入] -> Filter[数据处理] -> Output[结果输出]



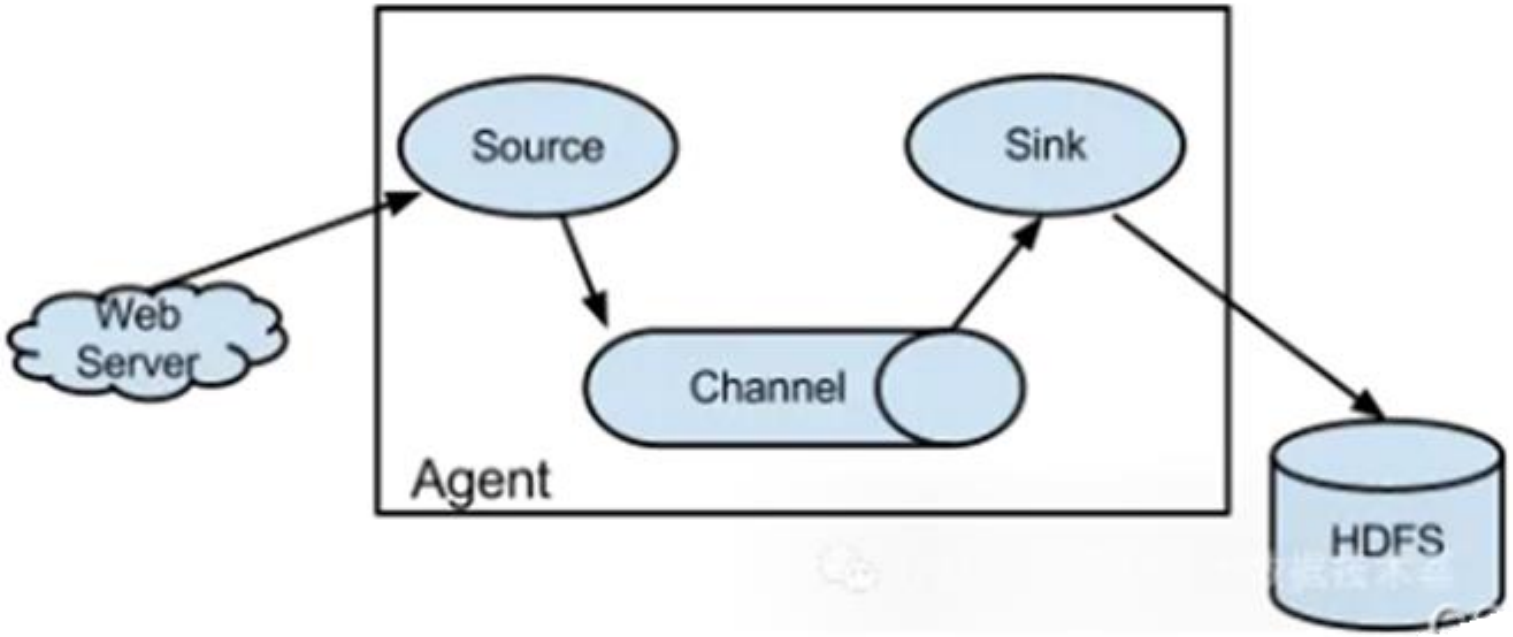
- 多个 Filter 构建了数据处理的 Pipeline，满足各种各样的数据处理需求，可以直接通过 SQL 构建数据处理的 Pipeline，简单高效。目前 seatunnel 支持的Filter列表, 仍然在不断扩充中。您也可以开发自己的数据处理插件，整个系统是易于扩展的
- **Input plugin**
  - Fake, File, Hdfs, Kafka, S3, Socket, 自行开发的 Input plugin
- **Filter plugin**
  - Add, Checksum, Convert, Date, Drop, Grok, Json, Kv, Lowercase, Remove, Rename, Repartition, Replace, Sample, Split, Sql, Table, Truncate, Uppercase, Uuid, 自行开发的Filter plugin
- **Output plugin**
  - Elasticsearch, File, Hdfs, Jdbc, Kafka, Mysql, S3, Stdout, 自行开发的 Output plugin

Apache SeaTunnel：生产应用案例

- **微博，增值业务部数据平台**
  - —微博某业务有数百个实时流式计算任务使用内部定制版 seatunnel，以及其子项目Guardian做seatunnel On Yarn 的任务监控
- **新浪，大数据运维分析平台**
  - —为新浪新闻、CDN 等服务做运维大数据的实时和离线分析，并写入Clickhouse
- **搜狗，搜狗奇点系统**
  - —搜狗奇点系统使用 seatunnel 作为 ETL 工具，帮助建立实时数仓体系
- **趣头条，趣头条数据中心**
  - —使用 seatunnel 支撑 mysql to hive 的离线 ETL 任务、实时 hiveto clickhouse 的 backfill 技术支撑，很好的 cover 离线、实时大部分任务场景
- **永辉超市子公司-永辉云创 会员电商数据分析平台**
  - —为永辉云创旗下新零售品牌永辉生活提供电商用户行为数据实时流式与离线SQL 计算
- **水滴筹数据平台**
  - —在 Yarn 上使用 seatunnel 做实时流式以及定时的离线批处理，每天处理 3～4T的数据量，最终将数据写入 Clickhouse

开源分布式、高可靠的流式日志采集系统： Apache Flume

- Apache Flume旨在可靠地、高效地从各种数据源采集、聚合和传输数据到目的地。Flume的设计目标是解决大规模数据采集的可靠性和扩展性问题。其基于可插拔的架构和配置驱动的方式，使得用户可以方便地定制和扩展数据采集的流程



Apache Flume：核心组件

Source（数据源）

- Flume的数据源是指数据采集的起点，它负责从外部数据源读取数据并将其传递给Flume的通道。Flume提供了多种数据源类型，例如Avro Source、Thrift Source和Spooling Directory Source。Avro Source支持通过Avro协议接收数据，Thrift Source支持通过Thrift协议接收数据，而Spooling Directory Source则监控指定目录下的文件，并将文件内容作为数据源

Channel（通道）

- 通道是Flume的核心组件之一，用于缓存和传递从数据源接收到的数据。Flume提供了多种通道类型，如Memory Channel、File Channel和Kafka Channel。Memory Channel将数据存储在内存中，适用于高吞吐量和低延迟的场景；File Channel将数据存储在本地文件系统中，适用于对数据持久化有要求的场景；Kafka Channel基于Apache Kafka实现，支持高可靠性和可扩展性

Sink（数据目的地）

- Sink是Flume的数据目的地，它负责将数据从通道中取出并发送到指定的目标系统。Flume提供了多种Sink类型，如HDFS Sink、Hive Sink和Elasticsearch Sink。HDFS Sink将数据写入Hadoop分布式文件系统，Hive Sink将数据写入Hive表，Elasticsearch Sink将数据写入Elasticsearch索引

十款开源数据集成工具对比

	社区活跃度	开源时间	主导者	可视化能力	执行引擎	开源协议	Apache顶级项目	推荐系数
InLong	Github：InLong Star：1.3k Fork：482 活跃度：比较活跃	2020.5.30	腾讯，腾讯云的数据集成平台基于InLong构建的	支持	Flink Pulsar/Kafka	Apache-2.0 license	是	☀☀☀☀☀
SeaTunnel（原名：Waterdrop）	Github：SeaTunnel Star：7.2k Fork：1.5k 活跃度：非常活跃	2018.1.6	乐视创建，目前主导者是白鲸开源科技	支持	支持 SeaTunnel Zeta、Flink、Spark 3 个引擎选其一作为运行时	Apache-2.0 license	是	☀☀☀☀☀
chunjun（FlinkX）	Github：chunjun Star：3.9k Fork：1.7k 活跃度：一般活跃	2018.4	袋鼠云	不支持	Flink	Apache-2.0 license	否	☀☀☀
BitSail	github地址：BitSail Star：1.6k Fork：323 活跃度：非常不活跃	2022.9.29	字节跳动	不支持	Flink 依赖Hadoop环境	Apache-2.0 license	否	☀
AirByte	github地址：AirByte Star：14k Fork：3.6 活跃度：非常活跃	2020.9.24	AirByte (专门做数据集成的公司，也有对应的企业版)	支持	自研引擎	MIT开源协议	否	☀☀☀

	社区活跃度	开源时间	主导者	可视化能力	执行引擎	开源协议	Apache顶级项目	推荐系数
Flume	github地址：Flume Star：1.6k Fork：323 活跃度：一般活跃	2010年	Flume 最初是 Cloudera 开发的日志收集系统，受到了业界的认可与广泛应用，后来逐步演化成支持任何流式数据收集的通用系统。	不支持	Source、Channel、Sink	Apache-2.0 license	是	☀☀☀
canal	github地址：Canal Star：1.6k Fork：323 活跃度：一般活跃	2012	阿里巴巴	不支持	client-server 模式，交互协议使用 protobuf 3.0	Apache-2.0 license	否	☀☀☀
Nifi	github地址：Nifi Star：4.4k Fork：2.6k 活跃度：非常活跃	2014	2006年NiFi由美国国家安全局（NSA）的Joe Witt创建	支持	基于DataFlow的编程模型构建的模块化的架构	Apache-2.0 license	是	☀☀☀☀
DataX	github地址：DataX Star：15.2k Fork：5.2k 活跃度：非常活跃	2015	阿里巴巴	开源版不支持，有其他人开源的webui，可以参考datax-webui	基于插件式的单机数据同步	Apache-2.0 license	否	☀☀☀☀

数据集成

- 基本概念
- 数据集成的方式
- 数据集成的架构
- 数据集成的工具
- 数据集成技术发展趋势

2024年数据集成技术现状：(1)多样化的数据源与存储系统

- 数据源变得更加多元化
  - —传统关系型数据库（如MySQL、PostgreSQL）
  - —NoSQL数据库（如MongoDB、Cassandra）
  - —分布式存储（如HDFS、Amazon S3）
  - —实时流数据（如Kafka）
  - —云服务平台（如AWS、Azure、Google Cloud）提供的多种数据源和存储系统也成为集成的重要组成部分
- **挑战**：如何高效管理和集成来自不同来源、不同格式的数据，成为数据集成工具面临的主要挑战
- **技术趋势**：Apache SeaTunnel等开源数据集成框架通过提供广泛的连接器支持，逐渐成为解决这一问题的重要工具。SeaTunnel目前支持的连接器包括数据库、文件系统、消息队列等，并且能够扩展自定义连接器来支持更多场景

2024年数据集成技术现状：(2)实时数据集成与流处理的普及

- 随着业务需求的变化，越来越多的行业需要实时数据处理能力。传统的批处理模式无法满足对低延迟和高吞吐量的需求，实时数据集成（如流数据处理）成为新的技术方向
- **技术现状**：流处理技术已经成为数据集成的一个关键趋势。Apache Kafka、Apache Pulsar和Apache Flink等流处理框架逐渐被集成到数据集成平台中，实现实时数据的抽取、转换和加载（ETL）操作
- **挑战**：实时数据集成需要在保证低延迟的同时，确保系统的高可用性和数据一致性。如何平衡这些需求仍是一个技术难题
- **技术趋势**：预计未来几年，基于事件驱动架构（EDA）的数据集成工具将会进一步发展，支持更大规模的数据流处理

2024年数据集成技术现状：(3)数据质量管理的核心地位

- 随着数据量的爆炸性增长，数据质量管理变得愈发重要。数据集成不仅仅是将数据从源系统传输到目标系统，还包括对数据的清洗、转换和校验，以确保数据的准确性、完整性和一致性
- **技术现状**：数据质量管理成为数据集成过程的关键组成部分。许多数据集成平台已经内置了数据清洗、去重、异常检测和格式校验等功能
- **挑战**：在海量数据的处理中，如何确保高效的数据质量监控与修复，尤其是在实时数据集成场景中，仍是一个需要突破的技术瓶颈
- **技术趋势**：随着AI和机器学习的发展，未来的数据集成工具可能会实现自动化的数据质量修复，利用智能算法发现并修复数据质量问题

2024年数据集成技术现状：(4)云原生数据集成的兴起

- 云计算的普及让企业越来越倾向于采用云原生架构，云原生数据集成技术逐渐成为主流
- **技术现状**：云平台原生的ETL工具和数据集成服务如AWS Glue、Azure Data Factory、Google Cloud Dataflow等开始得到广泛应用
- **挑战**：虽然云原生平台提供了强大的弹性和扩展性，但在多云环境和数据隐私保护方面，企业仍面临挑战
- **技术趋势**：未来，数据集成工具将更加注重跨云和混合云环境的支持，并加强与云原生服务的深度集成，提升跨平台的数据处理能力